

Java Collections Framework

A Complete Study Guide — Basic to Advanced

Covers: Core interfaces • List, Set, Map, Queue, Deque • Iterators • Comparable & Comparator • Collections utility class • Concurrent collections • Time complexity • Interview Q&A

Table of Contents

- Part 1 — Foundations: What Collections Are and Why They Matter
- Part 2 — The Core Interfaces and Class Hierarchy
- Part 3 — The List Interface (ArrayList, LinkedList, Vector)
- Part 4 — The Set Interface (HashSet, LinkedHashSet, TreeSet)
- Part 5 — The Map Interface (HashMap, LinkedHashMap, TreeMap, Hashtable)
- Part 6 — Queues and Deques (ArrayDeque, PriorityQueue, LinkedList)
- Part 7 — Iterating: Iterator, ListIterator, and the Enhanced For-Loop
- Part 8 — Ordering: Comparable vs Comparator
- Part 9 — The Collections and Arrays Utility Classes
- Part 10 — Time Complexity Cheat Sheet
- Part 11 — Advanced Topics: Concurrency, Fail-Fast, Generics, Streams
- Part 12 — Common Pitfalls and Interview Questions

PART 1

Foundations

What collections are, and why the framework exists

1.1 What is a Collection?

A **collection** is an object that groups multiple elements into a single unit. You use collections to store, retrieve, manipulate, and communicate aggregate data — instead of manually managing arrays, which have a fixed size and few built-in operations.

1.2 What is the Collections Framework?

The **Java Collections Framework (JCF)** is a unified architecture (introduced in Java 1.2) for representing and manipulating collections. It consists of three main parts:

- **Interfaces** — abstract data types like List, Set, Map, Queue that define behavior.
- **Implementations** — concrete classes like ArrayList, HashMap that implement those interfaces.
- **Algorithms** — static utility methods (in the Collections class) like sorting and searching that work on any implementation of the interfaces.

1.3 Why Use Collections Instead of Arrays?

Feature	Array	Collection
Size	Fixed at creation	Grows/shrinks dynamically
Data types	Primitives + objects	Objects only (autoboxing for primitives)
Built-in methods	Very few (length only)	Rich API: add, remove, search, sort, etc.
Type safety	Yes, at compile time	Yes, via generics
Memory	Contiguous, efficient	Slight overhead, but far more flexible

Note: Arrays are still useful for fixed-size, performance-critical, or primitive-heavy data. Collections are preferred whenever the size is unknown or changes at runtime.

PART 2

Core Interfaces

The hierarchy every implementation is built on

2.1 The Interface Hierarchy

At the very top sits **Iterable**. Everything that can be used in a for-each loop implements it. **Collection** extends **Iterable**, and three major sub-interfaces branch from it: **List**, **Set**, and **Queue**. **Map** is a separate hierarchy — it is *not* a **Collection**, because it stores key-value pairs rather than single elements.

```
Iterable
  ■■■ Collection
    ■■■ List          (ordered, duplicates allowed)
      ■ ■■■ ArrayList
      ■ ■■■ LinkedList
      ■ ■■■ Vector / Stack
    ■■■ Set          (no duplicates)
      ■ ■■■ HashSet
      ■ ■■■ LinkedHashSet
      ■ ■■■ SortedSet -> NavigableSet -> TreeSet
    ■■■ Queue        (FIFO / priority ordering)
      ■■■ LinkedList
      ■■■ PriorityQueue
      ■■■ Deque -> ArrayDeque

Map (separate hierarchy – key/value pairs)
  ■■■ HashMap
  ■■■ LinkedHashMap
  ■■■ Hashtable
  ■■■ SortedMap -> NavigableMap -> TreeMap
```

2.2 Key Interface Contracts

Interface	Ordering	Duplicates	Null allowed
List	Insertion order (index based)	Yes	Yes (multiple)
Set	Depends on implementation	No	Depends (HashSet: 1 null)
Queue	FIFO or priority order	Usually yes	Mostly no (PriorityQueue: no)
Deque	Insertion order, both ends	Yes	No
Map	Depends on implementation	Keys: No / Values: Yes	Depends (HashMap: 1 null key)

PART 3

The List Interface

ArrayList, LinkedList, and Vector

3.1 ArrayList

Backed by a resizable array. Best for frequent random access (get/set by index); worse for frequent inserts/deletes in the middle since elements must shift.

```
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.add(1, "Meera");    // insert at index 1
names.get(0);             // "Asha"
names.remove("Ravi");     // remove by value
names.set(0, "Asha K.");  // replace at index
System.out.println(names); // [Asha K., Meera]
```

Key characteristics

- Default capacity 10; grows by 50% when full (new capacity = old + old/2).
- get(index) is O(1) — direct array access.
- add(E) at the end is amortized O(1); add(index, E) is O(n) due to shifting.
- Not synchronized — not thread-safe by default.

3.2 LinkedList

A doubly-linked list implementation of both List and Deque. Good for frequent insertions/deletions at the ends or middle (once positioned), poor for random access.

```
LinkedList<Integer> ll = new LinkedList<>();
ll.addFirst(10);
ll.addLast(20);
ll.getFirst();    // 10
ll.removeLast();  // removes 20
// Also usable as a Queue or Stack:
ll.offer(5);      // add to tail
ll.poll();        // remove from head
```

Key characteristics

- get(index) is O(n) — must traverse from head or tail.
- addFirst/addLast/removeFirst/removeLast are O(1).
- Uses more memory per element than ArrayList (stores next/prev pointers).

3.3 Vector and Stack (Legacy)

Vector is a synchronized, legacy equivalent of ArrayList (grows by 100% by default, doubling in size). **Stack** extends Vector and adds push/pop/peek for LIFO behavior. Both predate the Collections Framework and are rarely used in modern code — prefer ArrayList, or ArrayDeque for stack behavior.

```
Deque<Integer> stack = new ArrayDeque<>(); // preferred over java.util.Stack
stack.push(1);
stack.push(2);
stack.pop();    // 2 (LIFO)
```


PART 4

The Set Interface

HashSet, LinkedHashSet, TreeSet

4.1 HashSet

Backed by a HashMap internally. Offers $O(1)$ average time for add/remove/contains but gives **no guarantee of iteration order**.

```
Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("collections");
tags.add("java");    // ignored – duplicate
tags.contains("java"); // true
tags.size();         // 2
```

4.2 LinkedHashSet

Extends HashSet but maintains a doubly-linked list running through all entries, so iteration happens in **insertion order**. Slightly more overhead than HashSet.

4.3 TreeSet

Implements NavigableSet, backed by a Red-Black tree (a self-balancing BST). Elements are always kept in **sorted order** — either their natural ordering (Comparable) or a supplied Comparator. add/remove/contains are $O(\log n)$.

```
TreeSet<Integer> scores = new TreeSet<>();
scores.add(50);
scores.add(10);
scores.add(30);
System.out.println(scores);    // [10, 30, 50] -- always sorted
scores.first();               // 10
scores.last();                // 50
scores.higher(10);            // 30 (smallest element strictly greater than 10)
scores.floor(25);             // 10 (largest element <= 25)
```

Choosing a Set implementation

Implementation	Order	Null	add/contains	Backed by
HashSet	Unordered	One null allowed	$O(1)$ avg	HashMap
LinkedHashSet	Insertion order	One null allowed	$O(1)$ avg	Hash table + linked list
TreeSet	Sorted order	No null (usually)	$O(\log n)$	Red-Black tree

PART 5

The Map Interface

HashMap, LinkedHashMap, TreeMap, Hashtable

5.1 HashMap

Stores key-value pairs using a hash table. Average $O(1)$ for get/put/remove. Allows one null key and multiple null values. Not thread-safe. Iteration order is unspecified and can change after resizing.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Asha", 28);
ages.put("Ravi", 32);
ages.get("Asha");           // 28
ages.getDefault("Sam", 0); // 0 (key not present)
ages.containsKey("Ravi");   // true
ages.remove("Ravi");
for (Map.Entry<String, Integer> e : ages.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

How HashMap works internally

- Keys are hashed via `hashCode()`, then bucketed into an internal array (table).
- Collisions (two keys, same bucket) are handled with a linked list of entries.
- Since Java 8: if a bucket's linked list grows beyond 8 entries (and table size ≥ 64), it converts to a red-black tree for $O(\log n)$ worst case instead of $O(n)$.
- Default initial capacity is 16, default load factor is 0.75 — the table resizes (doubles) once size exceeds $\text{capacity} \times \text{load factor}$.
- Correct `equals()` and `hashCode()` overrides on key objects are essential; inconsistent implementations break lookups.

5.2 LinkedHashMap

Maintains insertion order (or optionally access order, useful for building an LRU cache by overriding `removeEldestEntry()`).

```
// Simple LRU cache using LinkedHashMap
Map<Integer, String> lru = new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
        return size() > 3; // capacity of 3
    }
};
```

5.3 TreeMap

Implements `NavigableMap`; keys are always sorted (natural order or a supplied `Comparator`). Backed by a Red-Black tree. `get/put/remove` are $O(\log n)$.

```
TreeMap<String, Integer> map = new TreeMap<>();
map.put("banana", 3);
map.put("apple", 5);
map.firstKey(); // "apple"
map.lastKey();  // "banana"
map.ceilingKey("b"); // "banana" (smallest key >= "b")
```

5.4 Hashtable (Legacy)

A synchronized, legacy version of HashMap. Does not allow null keys or values. Rarely used today — prefer HashMap for single-threaded code and ConcurrentHashMap for multi-threaded code.

5.5 Choosing a Map

Implementation	Order	Null key/value	get/put	Thread-safe
HashMap	Unordered	1 null key, many null values	O(1) avg	No
LinkedHashMap	Insertion/access order	1 null key, many null values	O(1) avg	No
TreeMap	Sorted by key	No null key (Comparable)	O(log n)	No
Hashtable	Unordered	No nulls	O(1) avg	Yes (synchronized)
ConcurrentHashMap	Unordered	No null key/value	O(1) avg	Yes (fine-grained)

PART 6

Queues and Deques

FIFO, LIFO, and priority-based structures

6.1 Queue Basics

A Queue typically orders elements FIFO (first-in-first-out). Common implementations: LinkedList, ArrayDeque, and PriorityQueue.

Operation	Throws exception	Returns special value
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code>
Examine	<code>element()</code>	<code>peek()</code>

6.2 ArrayDeque

A resizable array implementation of Deque (double-ended queue). Faster than LinkedList for stack and queue operations, and generally the recommended choice for both use cases in modern Java. Does not allow null elements.

```
Deque<Integer> dq = new ArrayDeque<>();
dq.addFirst(1);
dq.addLast(2);
dq.pollFirst();    // 1
dq.pollLast();     // 2
// As a stack:
dq.push(10); dq.push(20); dq.pop(); // 20
```

6.3 PriorityQueue

A binary heap based queue where elements are ordered by natural ordering or a supplied Comparator — **not** insertion order. The head is always the smallest element (min-heap by default). `offer/poll` are $O(\log n)$; `peek` is $O(1)$.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5); pq.offer(1); pq.offer(3);
pq.poll();    // 1 (smallest first)

// Max-heap using a comparator:
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
maxHeap.offer(5); maxHeap.offer(1); maxHeap.offer(3);
maxHeap.poll(); // 5
```

Note: Iterating a PriorityQueue does NOT return elements in sorted order — only `poll()` guarantees the smallest (or highest-priority) element comes out first.

PART 7

Iterating Collections

Iterator, ListIterator, and for-each

7.1 The Iterator Interface

Every Collection can produce an Iterator via `iterator()`. It supports safe removal during traversal — using a collection's own `remove()` while iterating with a for-each loop throws a `ConcurrentModificationException`.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    int n = it.next();
    if (n % 2 == 0) {
        it.remove();    // SAFE removal during iteration
    }
}
System.out.println(nums);    // [1, 3, 5]
```

7.2 ListIterator

Available only on List implementations. Extends Iterator to support bidirectional traversal and in-place modification (`set/add`) during iteration.

```
ListIterator<String> lit = names.listIterator();
while (lit.hasNext()) {
    String s = lit.next();
    if (s.equals("Ravi")) {
        lit.set("Ravi K.");    // replace in place
    }
}
while (lit.hasPrevious()) {
    System.out.println(lit.previous());    // traverse backwards
}
```

7.3 Fail-Fast vs Fail-Safe Iterators

- **Fail-fast** (`ArrayList`, `HashMap`, `HashSet`, etc.): detects structural modification during iteration using an internal `modCount`, and throws `ConcurrentModificationException` immediately.
- **Fail-safe** (`CopyOnWriteArrayList`, `ConcurrentHashMap`): iterate over a snapshot or a weakly-consistent view, so concurrent modification during iteration does not throw — but the iterator may not reflect the very latest changes.

PART 8

Ordering

Comparable vs Comparator

8.1 Comparable — Natural Ordering

Implemented by the class whose instances are being compared. Defines a single, "natural" ordering via `compareTo()`. Used automatically by `Collections.sort()`, `TreeSet`, `TreeMap`.

```
public class Employee implements Comparable<Employee> {
    String name;
    int salary;

    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary); // ascending by salary
    }
}

List<Employee> list = ...;
Collections.sort(list); // uses compareTo()
```

8.2 Comparator — External, Pluggable Ordering

A separate object that defines an ordering independent of the class itself. Useful when you need multiple orderings, or the class isn't yours to modify.

```
// Classic anonymous class:
Collections.sort(list, new Comparator<Employee>() {
    public int compare(Employee a, Employee b) {
        return a.name.compareTo(b.name);
    }
});

// Modern lambda + Comparator chaining:
list.sort(Comparator.comparing((Employee e) -> e.name)
    .thenComparing(e -> e.salary));

// Reverse order:
list.sort(Comparator.comparingInt((Employee e) -> e.salary).reversed());
```

8.3 Comparable vs Comparator — Quick Comparison

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	<code>compareTo(T o)</code>	<code>compare(T a, T b)</code>
Location	Inside the class being compared	External, separate class or lambda
Number of orderings	One (natural order)	Many (as needed)
Typical use	Default sort order for a type	Custom / multiple sort criteria

PART 9

Utility Classes

Collections and Arrays

9.1 The Collections Class

`java.util.Collections` provides static algorithms that operate on or return collections.

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1));
Collections.sort(nums);                // [1, 3, 5, 8]
Collections.reverse(nums);             // [8, 5, 3, 1]
Collections.max(nums);                 // 8
Collections.min(nums);                 // 1
Collections.shuffle(nums);              // random order
Collections.frequency(nums, 3);         // count of value 3
Collections.unmodifiableList(nums);    // read-only view
Collections.synchronizedList(nums);    // thread-safe wrapper
List<Integer> empty = Collections.emptyList();
List<Integer> single = Collections.singletonList(42);
```

9.2 The Arrays Class

`java.util.Arrays` offers similar utilities for raw arrays, plus conversion helpers to bridge arrays and collections.

```
int[] arr = {5, 3, 8, 1};
Arrays.sort(arr);                      // [1, 3, 5, 8]
int idx = Arrays.binarySearch(arr, 5); // works only on sorted arrays
List<Integer> list = Arrays.asList(1, 2, 3); // fixed-size list view (backed by array!)
int[] copy = Arrays.copyOf(arr, 6);      // pads with 0s
System.out.println(Arrays.toString(arr));
```

Note: `Arrays.asList()` returns a fixed-size list backed by the original array — you can `set()` elements but calling `add()` or `remove()` throws `UnsupportedOperationException`.

9.3 Immutable / Factory Collections (Java 9+)

```
List<Integer> l = List.of(1, 2, 3);
Set<String> s = Set.of("a", "b");
Map<String, Integer> m = Map.of("x", 1, "y", 2);
// All of these are truly immutable – add/remove/set throws UnsupportedOperationException
```

PART 10

Time Complexity

The cheat sheet you actually need to memorize

10.1 List Implementations

Operation	ArrayList	LinkedList
get(index)	O(1)	O(n)
add(end)	O(1) amortized	O(1)
add(index)	O(n)	O(n) (O(1) once positioned)
remove(index)	O(n)	O(n)
contains / indexOf	O(n)	O(n)

10.2 Set Implementations

Operation	HashSet	LinkedHashSet	TreeSet
add / remove / contains	O(1) avg	O(1) avg	O(log n)
Iteration order	Undefined	Insertion order	Sorted order

10.3 Map Implementations

Operation	HashMap	LinkedHashMap	TreeMap
get / put / remove	O(1) avg	O(1) avg	O(log n)
Iteration order	Undefined	Insertion order	Sorted by key

10.4 Queue / Deque Implementations

Operation	ArrayDeque	PriorityQueue	LinkedList
offer / poll (ends)	O(1) amortized	O(log n)	O(1)
peek	O(1)	O(1)	O(1)

PART 11

Advanced Topics

Concurrency, generics, and streams

11.1 Concurrent Collections

`java.util.concurrent` provides thread-safe collections designed for high-concurrency access, generally outperforming synchronized wrappers under contention.

Class	Purpose
<code>ConcurrentHashMap</code>	Thread-safe map with fine-grained locking (segment/bucket level); no null keys/values.
<code>CopyOnWriteArrayList</code>	Thread-safe list; every mutation copies the underlying array. Great for read-heavy, write-rare scenarios.
<code>BlockingQueue</code> (interface)	Queue that blocks on offer/poll when full/empty — the backbone of producer-consumer patterns.
<code>ConcurrentLinkedQueue</code>	Lock-free, thread-safe unbounded FIFO queue based on linked nodes.
<code>ArrayBlockingQueue</code> / <code>LinkedBlockingQueue</code>	Bounded/unbounded blocking queue implementations, common in thread pools.

```
// Thread-safe counter map
Map<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge("apple", 1, Integer::sum); // atomic increment

// Producer-consumer with a blocking queue
BlockingQueue<String> queue = new LinkedBlockingQueue<>(100);
queue.put("task"); // blocks if full
String task = queue.take(); // blocks if empty
```

11.2 Generics in Collections

Generics give compile-time type safety, eliminating manual casting and `ClassCastException` risk at runtime.

```
// Bounded wildcard: read from any list of Number or its subtypes
public static double sum(List<? extends Number> list) {
    double total = 0;
    for (Number n : list) total += n.doubleValue();
    return total;
}

// Bounded wildcard: write Integers into any list of Integer or its supertypes
public static void fill(List<? super Integer> list) {
    list.add(1);
    list.add(2);
}
```

Note: Mnemonic: PECS — "Producer Extends, Consumer Super". Use `? extends T` when you only read from the structure; use `? super T` when you only write to it.

11.3 Collections and the Stream API

Since Java 8, collections integrate directly with streams for declarative transformation, filtering, and aggregation.

```
List<String> names = List.of("Asha", "Ravi", "Meera", "Aman");

List<String> filtered = names.stream()
    .filter(n -> n.startsWith("A"))
    .sorted()
    .collect(Collectors.toList());
// [Aman, Asha]

Map<Character, List<String>> grouped = names.stream()
    .collect(Collectors.groupingBy(n -> n.charAt(0)));
// {A=[Asha, Aman], R=[Ravi], M=[Meera]}

long count = names.stream().filter(n -> n.length() > 4).count();
```

11.4 equals() and hashCode() Contract

Hash-based collections (HashMap, HashSet) rely on a strict contract: if two objects are equal per equals(), they must return the same hashCode(). Violating this silently breaks lookups — an object may become unfindable even though an "equal" copy exists in the set.

- If a.equals(b) is true, then a.hashCode() == b.hashCode() must also be true.
- The reverse is not required — different objects may share a hash code (a collision), which is handled internally.
- Always override both methods together, never just one.
- For keys used in HashMap/HashSet, prefer immutable objects — mutating a key after insertion can make it unfindable.

PART 12

Pitfalls & Interview Prep

Common mistakes and frequently asked questions

12.1 Common Pitfalls

- **Modifying a list while iterating with for-each** — throws `ConcurrentModificationException`. Use an explicit iterator and `it.remove()`, or `removeIf()`.
- **Using `Arrays.asList()` and expecting add/remove to work** — it's a fixed-size view; wrap it: `new ArrayList<>(Arrays.asList(...))`.
- **Forgetting to override `equals()`/`hashCode()`** for custom objects used as `HashMap` keys or in a `HashSet` — lookups silently fail.
- **Using a mutable object as a `HashMap` key, then mutating it** — its hash bucket becomes stale and the entry becomes unreachable.
- **Assuming `HashMap` iteration order is stable** — it isn't; use `LinkedHashMap` if order matters.
- **Comparing with `==` instead of `equals()`** for wrapper types like `Integer` beyond the cached range (-128 to 127), which can produce surprising results.
- **Not specifying initial capacity** for a `HashMap`/`ArrayList` when the final size is roughly known — this avoids repeated resizing and rehashing.

12.2 Frequently Asked Interview Questions

Q1. Why is Map not part of the Collection interface?

Because `Collection` represents a group of single elements, while `Map` represents a group of key-value pairs. Their contracts (`add(E)` vs `put(K,V)`) are fundamentally different, so unifying them would break type safety and clarity.

Q2. How does HashMap handle collisions?

Colliding keys land in the same bucket and are stored in a linked list. Since Java 8, if a bucket's list grows beyond 8 entries (and the table is large enough), it's converted to a red-black tree, improving worst-case lookup from $O(n)$ to $O(\log n)$.

Q3. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally modified during iteration. Fail-safe iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) operate on a snapshot or weakly consistent view and don't throw, though they may not reflect the latest updates.

Q4. Why should you override hashCode() whenever you override equals()?

Hash-based collections rely on the contract that equal objects must have equal hash codes. Breaking this means an object can be "lost" inside a `HashMap`/`HashSet` — added successfully but never found again via `contains()` or `get()`.

Q5. When would you choose LinkedList over ArrayList?

When your workload involves frequent insertions/removals at the head, tail, or a known iterator position, and you rarely need random access by index. In practice, `ArrayDeque` often outperforms `LinkedList` for pure queue/stack use cases.

Q6. What's the difference between Comparable and Comparator?

Comparable defines a single natural ordering inside the class itself (`compareTo`). Comparator defines an external, swappable ordering (`compare`), letting you sort the same type in multiple different ways without modifying the class.

Q7. How does ConcurrentHashMap avoid locking the entire map?

It splits the underlying table into segments/buckets and applies fine-grained locking (or, in modern versions, CAS-based operations) only where a write actually happens, letting many threads read and write different regions concurrently.